# Prática 05 — TDD em Java

João Gabriel De Souza Madeira

RA:32516329

# RESPOSTAS.md

### 1. No Ciclo 1 usamos `return 70`. Por que isso é aceitável no TDD? O que obrigou a escrever a soma de verdade?

- **Por que é aceitável:** No TDD (Test-Driven Development), seguimos a regra de escrever apenas o código necessário para fazer o teste passar (**Fase Verde**). Retornar um valor fixo (`return 70`) é uma técnica conhecida como *fake it* (fingir até conseguir). O objetivo nessa fase inicial é validar se o mecanismo de testes está funcionando e dar o primeiro passo de forma simples, sem complexidade prematura.
- **O que obrigou a escrever a soma real:** O que quebrou essa "trapaça" foi o **Ciclo 2 (Triangulação)**, onde introduzimos um segundo teste com valores diferentes (ex: somar `10, 15, 25` esperando `50`). Como o método continuava retornando fixo `70`, o novo teste falhou, nos obrigando a substituir o valor fixo pela lógica real de soma (`return a1 + a2 + a3;`).

### 2. No Ciclo 3, por que testar com 69 e 70, e não com 50 e 90?

Porque 69 e 70 representam os **valores limites (análise de valor limite)** da regra de negócio (a nota mínima de aprovação que é 70). Alguém distraído poderia acidentalmente programar a condição como `>` (maior que) em vez de `>=` (maior ou igual). Se testássemos apenas com 50 (reprovado) e 90 (aprovado), ambos passariam tranquilamente mesmo com o operador errado. O valor 70 é exatamente a fronteira: se a lógica estiver errada (`> 70`), o teste com 70 falha imediatamente, garantindo a robustez da validação.

### 3. No Ciclo 4, o que mudou na refatoração? Como você teve certeza de que não quebrou nada?

- **O que mudou:** A refatoração eliminou a duplicação de códigos repetidos. Em vez de termos vários blocos `if` idênticos espalhados para validar cada nota individualmente, extraímos essa lógica para um método privado reutilizável (`validarNota`), aplicando também o uso de constantes (`NOTA_MAXIMA_A1`, etc.) para remover números mágicos.
- **Certeza de que nada quebrou:** Tivemos certeza absoluta rodando a **suíte completa de testes unitários** após a alteração. Como todos os testes permaneceram verdes (passando), comprovamos que a estrutura interna do código

mudou, mas o comportamento externo (as regras de negócio e validações) continuou intacto.

## 4. No Ciclo 8, por que alteramos os testes antes de mudar o código?

Porque no TDD os **testes funcionam como a especificação viva do sistema**. Quando os requisitos de negócio mudam (como a secretaria exigir mensagens detalhadas de motivo de reprovação), o comportamento esperado do sistema muda primeiro. Alterar os testes antes reflete essa nova regra, garantindo que o código de produção só seja modificado para atender estritamente ao novo comportamento especificado, mantendo o ciclo rigoroso de passar do Vermelho para o Verde.

## 5. Qual ciclo foi o mais difícil para você, e por quê?

*(Sugestão de resposta pessoal — você pode adaptar conforme a sua experiência)*: O ciclo mais desafiador foi o **Ciclo 6 ou 7**, pois exigiram combinar múltiplas regras lógicas simultâneas (condições compostas com `&&`, `||`, verificação de frequências e regras de substituição de notas com `Math.min`). No entanto, quebrar esses requisitos em pequenos testes unitários passo a passo tornou a implementação muito mais controlada e segura do que tentar escrever tudo de uma vez.

Professor Reinaldo Carlos Mendes olha simplesmente não deu o meu código não rodou de jeito nenhum e não consegui executar ele. eu só consegui responder as respostas. md